

Redis Sorted Sets Explained

Try to understand Sorted Sets with a leaderboard example



Dunith Danushka

Follow

7 min read · Mar 17, 2021



118

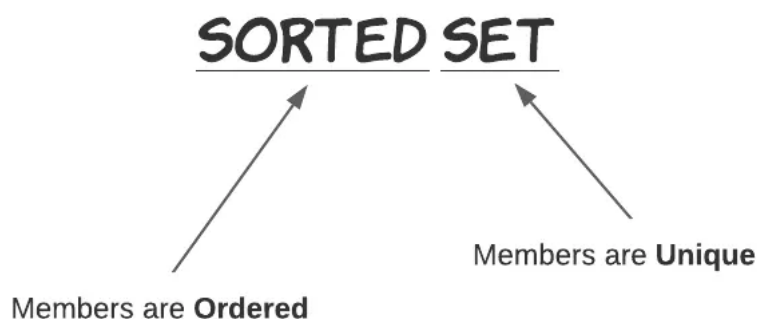


5



Photo by [Sigmund](#) on [Unsplash](#)

Before diving in, let's try to understand Sorted Sets by deconstructing the name.



As per the above, Sorted Set is a Set data structure that doesn't allow duplicate members. At the same time, its members are ordered in ascending order. By combining both, we can define a Sorted Set as an ordered collection of unique members.

Sorted Set is similar to the Set data structure in Redis. Members can be a list of non-repeating strings. The only difference is that each member is associated with a score, a floating-point number that provides a sorting order for the Sorted Set. Members are always sorted from the smallest to the greatest score.

The fitness leaderboard

The best way to understand a Sorted Set is to picture it as a table with score and member columns. The following is an imaginary fitness leaderboard where the member column represents the players while the score represents the number of steps they had walked so far.

SCORE	MEMBER		RANK
340	Dave	←	0
400	Kendra	←	1
560	Tom	←	2
650	Barbara	←	3
690	Jennifer	←	4
690	Peter	←	5
740	Frank	←	6

players

Players with their step counts

The leaderboard has been sorted in ascending order, based on the scores. Scores can be duplicated, but the members can not.

If two members have the same score, Redis sorts them based on members' lexicographical order. For example, **Jennifer** and **Peter** have the same score of 690. But Redis makes sure to put Jennifer before Peter because J comes before P.

Top highlight

You can also notice the **Rank** on the right side. Rank is an implicit attribute assigned to each member, based on the magnitude of its score. It's a zero-based index. The member with the smallest score will be assigned the rank of 0.

Rank is useful when you are retrieving members from the sorted set.

Benefits and use cases

Redis guarantees that a Sorted Set is always sorted. There is no additional sorting required for the client who retrieves the stored members back. That

saves CPU cycles in the client and also reduces the code complexity.

Sorted Sets allow you to add, remove, or update elements in a speedy way (in a time proportional to the logarithm of the number of elements). Since elements are taken in order and not ordered afterward, you can also get ranges by score or rank (position) quickly. Accessing the middle of a sorted set is also very fast, so you can use Sorted Sets as a smart list of non-repeating elements where you can quickly access everything you need.

That makes Sorted Sets an excellent choice of implementing real-time, low-latency leaderboards, priority queues, and secondary indexes.

So far, we have covered the basics of Sorted Sets. Let's look at some interesting operations we can perform on them.

Operations

We can perform operations on Sorted Sets to add, remove, increment, and retrieve members. Usually, these operations start with the letter 'Z.'

Let's use the fitness leaderboard example we discussed above to demonstrate these operations. Operations are performed using the Redis CLI. Note that I'm not going to cover all available Sorted Set operations here. I would instead glance through the operations that matter when building a leaderboard-like application. For a full list of operations, you can always refer to [Sorted Sets documentation](#).

Add new players and their steps to the game

When a new player joins, he/she should be added to the leaderboard with the number of steps.

The operation `ZADD` adds one more element to the Sorted Set. Below is the format of the command.

```
ZADD <key> <score> <member>
```

`key` is the name of the Sorted Set. The following command adds **Lester** with 790 steps to the `players`. In return, it gives you the number of elements added to the set, which is 1 in this case.

```
127.0.0.1:6379> ZADD players 790 Lester
(integer) 1
127.0.0.1:6379>
```

You can add any number of elements along with their scores as follows. Note that the elements are stored according to their score, not based on the order they've added.

```
127.0.0.1:6379> ZADD players 980 Mary 850 Alice
(integer) 2
127.0.0.1:6379>
```

After adding new players, the structure of `players` would look like this.

SCORE	MEMBER		RANK
340	Dave	←	0
400	Kendra	←	1
560	Tom	←	2
650	Barbara	←	3
690	Jennifer	←	4
690	Peter	←	5
740	Frank	←	6
790	Lester	←	7
850	Alice	←	8
980	Mary	←	9

players

With newly added Lester, Frank, and Mary

Increment/decrement player steps

When players perform their daily walks, their step count needs to be updated on the leaderboard.

You can use the `ZINCRBY` command to increment the score of a given member by any arbitrary value.

```
ZINCRBY <key> <increment> <member>
```

The following command increments **Tom's** steps by 10.

```
127.0.0.1:6379> ZINCRBY players 10 Tom
"570"
```

You can use a negative value to decrement a score. The following command decrements **Kendra's** steps by 5.

```
127.0.0.1:6379> ZINCRBY players -5 Kendra
"395"
```

Retrieve top 10 players by score

Now we need to retrieve only the top 10 players to display them on a leaderboard.

`ZREVRANGE` command makes that possible by returning only a specified set of members based on their reverse order score.



The command takes the following format.

```
ZREVRANGE <key> <start> <stop> [WITHSCORES]
```

`ZREVRANGE` returns members in reversed-rank order, with scores ordered from high to low. You have to specify starting and ending rank index positions with a `start` and `stop` parameters.

Let's get the top 10 players by the steps they had walked.

```
127.0.0.1:6379> ZREVRANGE players 0 9
1) "Mary"
2) "Alice"
3) "Lester"
4) "Frank"
5) "Peter"
6) "Jennifer"
7) "Barbara"
8) "Tom"
9) "Kendra"
10) "Dave"
```

There is one additional parameter: `WITHSCORES`, which returns the score for each member.

```
127.0.0.1:6379> ZREVRANGE players 0 9 WITHSCORES
1) "Mary"
2) "980"
3) "Alice"
4) "850"
5) "Lester"
6) "790"
7) "Frank"
8) "740"
9) "Peter"
10) "690"
11) "Jennifer"
12) "690"
13) "Barbara"
14) "650"
15) "Tom"
16) "570"
17) "Kendra"
18) "395"
19) "Dave"
20) "340"
```

The `ZRANGE` command does the opposite to this. It returns members in rank order, with scores ordered from **low to high**.

The following command returns the first ten players who had walked the least number of steps.

```
127.0.0.1:6379> ZRANGE players 0 9
1) "Dave"
2) "Kendra"
3) "Tom"
4) "Barbara"
5) "Jennifer"
6) "Peter"
7) "Frank"
8) "Lester"
9) "Alice"
10) "Mary"
```

Retrieve the rank and the score of an individual player

Now we need to find the score and the rank of a given player. Sometimes, that can be displayed on the player's profile.

`ZRANK` and `ZREVRANK` commands return the rank of a member, which is a 0 based index position.

The format of the command is:

```
[ZRANK|ZREVRANK] <key> <member>
```

The following command returns 0 for **Dave**, who's currently at the bottom of the leaderboard.

```
127.0.0.1:6379> ZRANK players Dave
(integer) 0
```

Similarly, `ZREVRANK` for **Mary** should return 0. Because it returns the rank in the reverse order, with scores sorted from low to high.

```
127.0.0.1:6379> ZREVRANK players Mary
(integer) 0
```

`ZSCORE` provides the current score associated with the member. The format of the command is:

```
ZSCORE <key> <member>
```

This returns **Peter's** score, which is 690.

```
127.0.0.1:6379> ZSCORE players Peter  
"690"
```

Get the player count based on the score

How can we find the number of players who had walked between a given range of steps? For example, how many players had walked more than 500 but less than 700 steps?

The command `ZCOUNT` will count the members between the min and the max score. That is inclusive. The format of the command is:

```
ZCOUNT <key> <min> <max>
```

The following command returns the number of players who had walked between 500 and 700 steps.

```
127.0.0.1:6379> ZCOUNT players 500 700  
(integer) 4
```

Remove players from the game

When players leave the game, we need to remove them from the leaderboard.

The `ZREM` command removes one or more members from a Sorted Set. The command format is:

```
ZREM <key> <member> [<member> ...]
```

We can remove **Barbara** from the leaderboard as she decided to leave the game because she was tired.

```
127.0.0.1:6379> ZREM players Barbara  
(integer) 1
```

Finally, after all these operations, our leaderboard looks like this.

```
127.0.0.1:6379> ZREVRANGE players 0 -1 WITHSCORES
1) "Mary"
2) "980"
3) "Alice"
4) "850"
5) "Lester"
6) "790"
7) "Frank"
8) "740"
9) "Peter"
10) "690"
11) "Jennifer"
12) "690"
13) "Tom"
14) "570"
15) "Kendra"
16) "395"
17) "Dave"
18) "340"
127.0.0.1:6379> █
```

Conclusion

Due to its natural ordering and efficiency in accessing stored elements, Sorted Sets are getting increasingly popular for building applications like real-time leaderboards. They typically store more than a million elements in

Implementing the same functionality with a traditional database would always be challenging. Even if you manage to do that, feeding that to a dashboard will be a recipe for disaster.

Thus, an in-memory data structure such as Redis shines well here.

Resources

Data types - Redis

Strings are the most basic kind of Redis value. Redis Strings are binary safe, this means that a Redis string can...

redis.io

Redis

Programming

Data Structures

Leaderboard

Software Development



Published in Analytics Vidhya

78K followers · Last published Dec 16, 2025

Follow

Analytics Vidhya is a community of Generative AI and Data Science professionals. We are building the next-gen data science ecosystem
<https://www.analyticsvidhya.com>



Written by Dunith Danushka

2.7K followers · 273 following

Follow

Editor of Tributary Data. Technologist, Writer, Senior Developer Advocate at Redpanda. Opinions are my own.

Responses (5)



Ishu

What are your thoughts?



Deepankar Acharyya

Mar 11, 2025 (edited)



Great article. Any idea how this is implemented internally?



1

[Reply](#)



Adolfem Moe

Dec 31, 2025



Awesome article, that 's excatly the explanation that I was searching for.



[Reply](#)



Игорь Т

Jan 12, 2022



Nvm found it "zrange key -1 -1"



[Reply](#)

[See all responses](#)

More from Dunith Danushka and Analytics Vidhya



 In Tributary D... by Dunith Danush... · May 30, 2021

A Visual Introduction to Debezium

A story-based introduction to understanding what Debezium is, how it is made of, and ho...

 389  7



 In Analytics Vidh... by Leland Rober... · Mar 6, 2020

Understanding the Mel Spectrogram

(and Other Topics in Signal Processing)

 2.4K  31



 In Analytics Vidh... by Ashutosh Sa... · Mar 20, 2021

How to Handle Categorical Features

Hello Everyone!!! Welcome to my blog. If you are dealing with the data, I am 110% certain...

 277  1



 In Tributary Da... by Dunith Danush... · May 6, 2021

Azure Service Bus Essentials—Scheduled Messages

 103  1



[See all from Dunith Danushka](#)

[See all from Analytics Vidhya](#)

